INHA UNIVERSITY IN TASHKENT

COMPUTER SECURITY COURSE
PROJECT SUBMISSION

Instructor: Dr. Kirti Seth

Student Name: Javohirbek Xatamov
Student ID: U2210251

School of Computer and Information Engineering
Department of Computer Science

Submission Date: April 24, 2026

Table of Contents

Abstract

This report presents a comprehensive analysis of the XZ Utils supply chain attack (CVE-2024-3094), as described in the research paper "Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack" by Przymus and Durieux (2025). The attack, discovered on March 28, 2024, represents a landmark case in which malicious actors exploited not technical flaws alone, but the entire open-source software (OSS) development process itself. Over a period of 2.6 years, the attacker systematically built trust within the XZ Utils community, gradually became a trusted maintainer, and ultimately injected a backdoor into a critical Linux compression library, targeting OpenSSH servers. Through a mixed-methods analysis of Git commits, GitHub event logs, and mailing list archives, the authors reconstruct the attack timeline across five phases and catalog the software engineering practices weaponized during the attack. This report covers the problem statement, methodology, key technical concepts, results, and offers an independent critical review of the research. The case carries broad implications for OSS governance, contributor vetting, and supply-chain security.

1. Introduction

Open-Source Software (OSS) forms the foundation of the modern digital economy. Studies estimate that approximately 90% of modern applications contain open-source components, making the security of OSS not merely a software engineering concern but a matter of global digital security. Despite this critical dependence, many essential OSS projects are maintained by small teams or even single volunteers who balance project responsibilities with personal commitments.

This structural vulnerability was dramatically illustrated by the XZ Utils attack (CVE-2024-3094), discovered in March 2024. XZ Utils is a widely used compression library present in most Linux distributions. A sophisticated threat actor—referred to as the Attacker (A)—spent over two and a half years infiltrating the project, building credibility, gaining maintainership rights, and ultimately injecting a backdoor that targeted OpenSSH servers through a chain involving *systemd* and *liblzma*.

The paper under review, authored by Piotr Przymus and Thomas Durieux, provides the first holistic software engineering analysis of this attack. Rather than focusing solely on the technical backdoor mechanism, the authors examine how standard software engineering practices—community management, CI/CD configuration, GitHub migration, code reviews—were weaponized to achieve and sustain the attack.

The study addresses three research questions:

RQ1: What is the timeline of the XZ Utils attack?
RQ2: What software engineering practices were employed by the attackers?
RQ3: What are the consequences and impacts of the attack?

This report analyses each of these dimensions while offering an independent critical evaluation of the research's contributions and limitations.

2. Problem Statement

The core problem addressed by this research lies at the intersection of OSS sustainability and supply chain security. Modern software ecosystems are deeply dependent on OSS components, yet many critical projects suffer from chronic under-resourcing. The "bus factor"—the risk of a project collapsing if a single key contributor becomes unavailable—is dangerously low for many foundational libraries.

This creates a fertile environment for a novel class of threat: long-term social engineering of OSS maintainers. Unlike traditional software vulnerabilities that exploit technical flaws in code, this category of attack exploits the human and organisational dimensions of open-source development.

Key aspects of the problem include:

• Critical infrastructure dependence: XZ Utils is a dependency of *systemd*, which in turn is linked to OpenSSH in many Linux distributions. A compromise at this level can expose millions of servers.

• Under-resourced maintainers: The XZ Utils project was maintained by a single developer, who was susceptible to burnout and welcomed new contributors without deeply vetting them.

• Invisible attack surface: Traditional security tooling (static analysis, CVE scanners, code review) focuses on code. The attack exploited non-code contributions—translations, CI/CD setup, community management—areas that receive far less security.

• Absence of detection frameworks: At the time, no established framework existed for analyzing or detecting software supply chain infiltration of this nature, leaving the OSS community unprepared.

3. Motivation and Significance

The XZ Utils attack is significant for several interconnected reasons:

1. Scale of potential damage: The backdoor specifically targeted OpenSSH server daemons on Debian- and Red Hat-based Linux systems. Had it not been discovered by a PostgreSQL developer (referred to as the Incident Observer, IO) noticing anomalous CPU usage during SSH logins, the backdoor could have granted arbitrary remote access—before authentication—to an attacker holding a private key. The potential for mass server compromise was enormous.

2. A new attack paradigm: Previous high-profile OSS security incidents (Heartbleed, Log4Shell) involved accidental vulnerabilities in code. The XZ Utils attack represents a deliberate, patient, multi-year infiltration of the development process itself. This raises the threat model for OSS to an entirely new level.

3. Automation amplifies the risk: The authors note that the attacker used grammar checkers, translation software, and other automation tools to scale up contributions cheaply. As Large Language Models (LLMs) become more capable, the cost of mounting similar attacks will decrease further—making this research timely and urgently relevant.

4. Governance gap: The attack exposed that OSS governance models built on implicit trust and voluntary contribution are insufficient for projects with large downstream impact.

5. Community resilience: On the positive side, the incident galvanized the OSS community. New security tools were developed (distro-backdoor-scanner, backseat-signed), dependency reduction initiatives began across major projects, and governance discussions were revived.

4. Methodology / Algorithm Description

The authors employ a mixed-methods research design, combining qualitative and quantitative analysis across multiple data sources. The methodology is designed to provide a holistic view of the attack rather than focusing on any single dimension.

Data Sources:
The dataset comprises four primary components:
• Git Repository Analysis: 1,020 commits with 91,263 automatically annotated lines, categorised by type (source code, documentation, test cases, translations) using the *PatchScope* tool.
• GitHub Project Metrics: 2,944 GitHub events including issue tracking, pull request history, and event logs sourced from GitHub Archive.
• XZ Utils Mailing List Archives: 307 mailing list messages covering development communications prior to the GitHub migration.
• Security Context Analysis: Checks against the Have I Been *Pwned* database for 62 unique email addresses to evaluate potential compromise vectors.

Analytical Approach:
• Open Coding: Since no established framework existed for analysing OSS supply chain attacks of this nature, the authors manually annotated all recorded interactions (code reviews, emails, discussions) to develop a classification scheme from the ground up.
• Automated Annotation: The *PatchScope* tool was used to classify code changes at a granular level.
• Statistical Analysis: Used to validate hypotheses about the involvement of multiple accounts, examining cross-contributor activity patterns.
• Timeline Reconstruction: The attack was divided into five phases (P1–P5) based on first occurrences of new attacker behaviours or the introduction of attack-enabling functionality.

The authors acknowledge that the study is based on a single case study, which limits generalisability, but argue that the severity and uniqueness of the incident justify the in-depth approach.

5. Key Concepts and Technical Approach

Understanding the attack requires familiarity with several interconnected technical concepts:

1. The Dependency Chain:
XZ Utils provides *liblzma*, a compression library. Many Linux distributions patch OpenSSH to support *systemd's* sd_notify function, linking OpenSSH against *libsystemd*. *libsystemd* in turn depends on *liblzma*. This creates an indirect dependency: *OpenSSH* → *libsystemd* → *liblzma* (XZ Utils). By compromising *liblzma*, the attacker could inject code that executes within the OpenSSH server process.

2. The *ifunc* Resolver (GNU Indirect Function):
The GNU *ifunc* feature allows dynamic selection of function implementations at runtime (e.g., choosing an optimised CPU-specific code path). This feature was introduced to XZ Utils on June 22, 2023, ostensibly for a performance benefit. The attacker later exploited it to replace the *RSA_public_decrypt* function at runtime—the function responsible for validating RSA signatures in OpenSSH. This enabled authentication bypass.

3. The Backdoor Mechanism:
The malicious code examines the RSA public modulus (N value) in the RSA structure passed to *RSA_public_decrypt*. Since this value is controlled by the connecting SSH client, an attacker with the correct private key could either: (a) send arbitrary payloads executed before authentication, or (b) bypass SSH password authentication entirely.

4. Obfuscation via the Build System:
The malicious payload was not present in the Git repository. It was embedded in binary test files (a common practice for compression library testing) and extracted during the build process via an obfuscated shell script. The backdoor only existed in official distribution *tarballs*, not in the source tree—a deliberate design choice to evade scrutiny.

5. The Five-Phase Attack Timeline:
Phase P1 (Oct 2021): Initial patch proposals and code reviews to establish presence.
Phase P2 (Jan 2022): First accepted commits; coordinated pressure from puppet accounts for more maintainers.
Phase P3 (Dec 2022): GitHub migration; attacker creates the organisation and gains ownership.
Phase P4 (Jun 2023): Introduction of *ifunc* resolver; attacker assumes most maintenance tasks.
Phase P5 (Jan 2024): First backdoor commit; release of XZ 5.6.0 and 5.6.1 with backdoor.

6. Software Engineering Practices as Attack Vectors:
The attacker weaponized: community management, CI/CD setup, translations, documentation, build system changes, GitHub migration, website migration, code reviews,

and test expansion (to hide payloads). These collectively built credibility while serving the attack.

6. Results and Performance Evaluation

The study's results address each of the three research questions:

RQ1 — Attack Timeline:
The attack spanned 2.6 years (October 2021 – March 2024). The authors identified five phases of increasing attacker involvement. During phases P1–P3, activities were primarily trust-building; during P4–P5, attack-enabling functionality and malicious code were introduced. Only 8 of 1,020 commits were confirmed malicious—meaning 99.2% of the attacker's contributions appeared legitimate. The timing of the final release was strategically aligned with the Red Hat 9.4 release cycle (end of April 2024), maximizing propagation potential.

RQ2 — Software Engineering Practices:
The attacker employed 10 distinct categories of SE practice (see Table III in the paper): community management, CI/CD setup, core code contributions, code review participation, translation, build system changes, test expansion, GitHub migration, website migration, and mailing list engagement. Crucially, documentation and translation activities dominated over code contributions, representing a low-scrutiny, high-credibility strategy. The attacker also gradually displaced the primary maintainer's responsibilities across all these areas.

RQ3 — Impact:
• Immediate: The backdoor was caught before widespread exploitation. GitHub suspended both the attacker's and (temporarily) the primary maintainer's accounts. Debian and Red Hat rolled back to earlier XZ versions.
• Post-incident review: The primary maintainer reviewed approximately three years of commits; 8 were confirmed malicious. Several commits related to ifunc were also reverted as a precaution.
• Community response: XZ Utils became more active post-attack. New security tools were developed (distro-backdoor-scanner, backseat-signed). Major projects (*CMake*, *Systemd*, *GNU binutils*, *OpenSSH*) began reducing dependencies. The OSS community broadly increased focus on contributor vetting, reproducible builds, and multi-stakeholder governance.

7. Advantages and Limitations

Advantages of the Research:

• Holistic perspective: Unlike prior reports that focused on isolated aspects (backdoor mechanics, contributor behaviour), this study integrates qualitative and quantitative methods across all available data sources, providing a comprehensive view.

• Public dataset: The authors released their full dataset and scripts (GitHub: *przymusp*/XZ-Attack), enabling replication and further research by the community.

• Novel classification framework: The open-coding approach produced the first systematic taxonomy of software engineering practices used in OSS supply chain attacks—a valuable contribution given the lack of prior frameworks.

• Practical relevance: Findings directly inform actionable defences: contributor vetting, anomalous pattern detection, CI/CD hardening, and governance reform.

• Timely publication: Published in April 2025, the research is contemporaneous with ongoing community efforts to address the attack's aftermath.

Limitations of the Research:

• Single case study: The analysis is based on one attack. Findings may not generalise to projects with different characteristics (larger teams, more active governance, different ecosystems).

• No access to private communications: The study relies on publicly available data. Private IRC logs, direct messages, and off-platform communications were unavailable, potentially missing key aspects of the attacker's strategy.

• Hindsight bias: Analysing events after the attack is known makes suspicious patterns appear more obvious than they would have been at the time. The authors acknowledge this but it remains a structural limitation of retrospective analysis.

• Attribution ambiguity: The role of the Anomalous Committer (♂AC), who introduced the *ifunc* resolver, remains unresolved. The study cannot definitively establish whether this was a collaborator or an unwitting contributor.

• Generalisability of automation risk: The claim that LLMs will lower the cost of such attacks is speculative and not empirically validated within this study.

8. Critical Review (Student Analysis)

The paper makes a genuinely important contribution to the field of software supply chain security by framing the XZ Utils incident through a software engineering lens. Below is an independent critical assessment:

Strengths of the Approach:
The decision to use open coding rather than forcing the data into an existing framework is methodologically sound given the novelty of the attack. The authors' use of multiple data sources (Git, GitHub events, mailing lists, HIBP) adds triangulation and reduces reliance on any single evidence stream. The public dataset is an exemplary practice for reproducible security research.

Areas for Improvement:
1. Lack of comparative analysis: The paper would benefit from a structured comparison with other documented OSS infiltration attempts (e.g., the event-stream npm incident, the *ctx/phpass PyPI* attack). This would help determine which findings are specific to XZ Utils versus general patterns in OSS supply chain attacks.

2. Detection methodology gap: While the paper identifies the attack's phases retrospectively, it does not propose or evaluate concrete detection algorithms or metrics that could have flagged the attacker's behaviour in real time. Operationalising the findings into detection heuristics would substantially increase the paper's practical value.

3. The *ifunc* contributor ambiguity: The unresolved status of the Anomalous Committer is a significant loose end. A more rigorous statistical or behavioural analysis of this contributor's activity patterns could either strengthen or weaken the attribution hypothesis.

4. LLM risk claim: The assertion that LLMs will enable cheaper, more scalable versions of this attack is presented without empirical support. A controlled experiment or at minimum a structured threat modelling exercise would strengthen this claim considerably.

5. Psychological dimension underexplored: The primary maintainer's susceptibility to welcoming an attacker who offered to help was partly a function of burnout and social pressure. The paper touches on this but does not analyse the psychological dynamics in depth—an area that is critical for designing preventative community practices.

Comparison with Existing Methods:
Traditional supply chain defences (SBOM, code signing, dependency pinning) would not have prevented this attack because the malicious code was introduced through the legitimate release process by a trusted maintainer. This exposes a fundamental gap: current defences assume the threat is external, not an insider who spent years earning trust. The paper's framing makes a compelling case that the OSS community needs a new category of

defence focused on contributor behaviour and process integrity, not just code integrity.

Suggested Future Directions:
• Develop real-time anomaly detection tools for contributor behaviour (activity spikes, scope expansion, credential changes).
• Study the social dynamics of maintainer burnout and its relationship to security decisions.
• Investigate the effectiveness of multi-maintainer governance models in high-impact OSS projects.
• Empirically evaluate whether LLM-generated contributions are detectable by existing review processes.

9. Real-World Application

The XZ Utils attack has prompted concrete changes across the OSS ecosystem, and the paper's findings map directly onto several real-world security practices:

1. Software Bill of Materials (SBOM): Organisations now increasingly require SBOMs to track transitive dependencies. Had SBOM practices been widely mandated, the XZ Utils → *systemd* → *OpenSSH* dependency chain would have been explicitly documented, making the risk more visible.

2. Reproducible Builds: The backdoor was only present in release tarballs, not the Git source. Reproducible build systems (where any user can independently reproduce the exact build artifact from source) would have flagged this discrepancy, as the tarball and the Git source would not produce identical binaries.

3. Contributor Vetting Tools: Post-attack, tools like distro-backdoor-scanner were developed to audit Linux distribution packages for anomalies. The paper's taxonomy of attacker SE practices directly informs what these tools should look for: sudden scope expansion, unusual CI/CD changes, disproportionate non-code contributions.

4. Multi-Stakeholder Governance: Critical OSS projects like the Linux kernel and OpenSSL now emphasise multi-maintainer models, where no single individual can release code unilaterally. This architectural safeguard reduces the risk of insider attacks.

5. Funding for OSS Security: Initiatives like the *OpenSSF* (Open-Source Security Foundation) and government-level efforts (e.g., US CISA's memory safety roadmap) now provide funding and tooling for critical OSS infrastructure—directly addressing the under-resourcing vulnerability that made XZ Utils a target.

10. Future Scope

The XZ Utils attack opens several important research directions that the paper itself acknowledges:

1. Longitudinal threat modelling: Developing threat models specifically designed for long-term OSS infiltration, accounting for multi-year trust-building phases, is essential. Current threat models focus on point-in-time vulnerabilities.

2. Automated contributor behaviour analysis: Machine learning models trained on contributor activity patterns (commit frequency, scope, interaction patterns) could flag anomalous behaviour in real time. The dataset released by the authors provides a foundation for such work.

3. AI and LLM contributions in OSS: As LLMs become capable of generating high-quality code, documentation, and translations, the OSS community needs frameworks for attributing and auditing automated contributions. Should LLM-generated contributions be labelled? Should they count toward contributor credibility?

4. Psychological resilience of maintainers: Research into how burnout and social pressure influence security-critical decisions by OSS maintainers is largely absent. Human factors engineering and organisational psychology could inform better support systems and decision frameworks for maintainers under pressure.

5. Cross-ecosystem studies: Replicating the analysis methodology across other OSS ecosystems (*npm*, *PyPI*, *Maven*, *crates.io*) would reveal whether the XZ Utils patterns are universal or ecosystem-specific, strengthening or qualifying the paper's generalisations.

6. Regulatory frameworks: As OSS becomes critical national infrastructure, regulatory frameworks (similar to financial sector oversight) may need to evolve. Research into appropriate governance models that preserve OSS openness while enforcing minimum security standards is urgently needed.

11. References

[1] Przymus, P. & Durieux, T. (2025). Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. arXiv:2504.17473v1 [cs.SE].

[2] Canfora, G., Di Sorbo, A., Forootani, S., Pirozzi, A., & Visaggio, C. A. (2020). Investigating the vulnerability fixing process in OSS projects: Peculiarities and challenges. Computers & Security, 99, 102067.

[3] Coelho, J. & Valente, M. T. (2017). Why modern open source projects fail. ESEC/FSE 2017. ACM. https://doi.org/10.1145/3106237.3106246

[4] Dias, E., Meirelles, P., Castor, F., Steinmacher, I., Wiese, I., & Pinto, G. (2021). What makes a great maintainer of open source projects? ICSE 2021. IEEE.

[5] Dias, L. F., Steinmacher, I., & Pinto, G. (2018). Who drives company-owned OSS projects: internal or external members? Journal of the Brazilian Computer Society, 24(1), 16.

[6] Gasser, O., Holz, R., & Carle, G. (2014). A deeper understanding of SSH: Results from Internet-wide scans. NOMS 2014. IEEE.

[7] Guizani, M. et al. (2021). The long road ahead: Ongoing challenges in contributing to large OSS organizations and what to do. CSCW2 2021.

[8] Hissam, S. A., Plakosh, D., & Weinstock, C. (2002). Trust and vulnerability in open source software. IEE Proceedings-Software, 149(1), 47–51.

[9] Hunt, T. (2019). Have I Been Pwned. https://haveibeenpwned.com

[10] Lenarduzzi, V. et al. (2020). Open source software evaluation, selection, and adoption: a systematic literature review. SEAA 2020. IEEE.

[11] Lifshitz-Assaf, H. & Nagle, F. (2021). The digital economy runs on open source. Here's how to protect it. Harvard Business Review.

[12] Mouratidis, H., Giorgini, P., & Manson, G. (2005). When security meets software engineering: a case of modelling secure information systems. Information Systems, 30(8), 609–629.

[13] O'Neill, P. H. The Internet Runs on Free Open-Source Software. Who Pays to Fix It? MIT Technology Review. https://www.technologyreview.com/2021/12/17/1042692/log4j-internet-open-source-hacking/

[14] Payne, C. (2002). On the security of open source software. Information Systems Journal,

12(1), 61–78.

[15] Przymus, P., Narębski, J., Fejzer, M., & Stencel, K. (2024). PatchScope – A Modular Tool for Annotating and Analyzing Contributions. https://ncusi.github.io/PatchScope/

[16] The Register. (2020). What happens when the maintainer of a JS library downloaded 26m times a week goes to prison? https://www.theregister.com/2020/03/26/corejs_maintainer_jailed_code_release

[17] Scalco, S., Paramitha, R., Vu, D.-L., & Massacci, F. (2022). On the feasibility of detecting injections in malicious npm packages. ARES 2022.

[18] Tulili, T. R., Capiluppi, A., & Rastogi, A. (2023). Burnout in software engineering: A systematic mapping study. Information and Software Technology, 155, 107116.

[19] Wang, X., Sun, K., Batcheller, A., & Jajodia, S. (2019). Detecting "0-day" vulnerability: An empirical study of secret security patch in OSS. DSN 2019. IEEE.

[20] Zhou, X. et al. (2024). OSS Malicious Package Analysis in the Wild. arXiv:2404.04991.

[21] Zimmermann, M., Staicu, C.-A., Tenny, C., & Pradel, M. (2019). Small world with high risks: A study of security threats in the npm ecosystem. USENIX Security 2019.